Day 3 – Python Collections for AWS Data Engineering

Today we will learn:

1. List
2. List indexing
3. List slicing
4. Adding/removing data from a list
5. Looping through a list
6. Tuple
7. Set
8. Dictionary
9. Dictionary operations
10. List of dictionaries
11. Real AWS Data Engineering examples
12. Practice programs

---

1. Python List

A **list** is used when we want to store **multiple values inside one variable**.

For example, without a list:

```
file1 = "customers.csv"file2 = "orders.csv"file3 = "products.csv"
```

Instead of creating three variables, we can create one list:

```
files = ["customers.csv", "orders.csv", "products.csv"]print(files)
```

Output

```
['customers.csv', 'orders.csv', 'products.csv']
```

Here:

files → Variable name

```
["customers.csv",
```

"orders.csv",

"products.csv"] → List

customers.csv → First item

orders.csv → Second item

products.csv → Third item

In Data Engineering, this is useful because we may receive many files:

customers.csv

orders.csv

products.csv

sales.csv

Instead of maintaining separate variables, we can keep all file names in one list.

---

2. List Index

Every item inside a list has a position called an **index**.

Very important:

Python index starts from 0, not 1.

Example:

```
files = ["customers.csv", "orders.csv",
"products.csv"]print(files[0])print(files[1])print(files[2])
```

Output

customers.csv

orders.csv

products.csv

Understand it like this:

Index	Value
0	customers.csv
1	orders.csv
2	products.csv

So:

```
print(files[0])
```

means:

Go to the files list and give me the item available at index 0.

The answer is:

customers.csv

---

3. Negative Index

Python also allows us to access items from the end.

```
files = ["customers.csv", "orders.csv", "products.csv"]print(files[-
1])print(files[-2])
```

Output

products.csv

orders.csv

Here:

-1 → Last item

-2 → Second last item

This can be useful when we want the latest/last item in a list.

---

4. Find Number of Items – len()

```
len() tells us how many items are available.
files = ["customers.csv", "orders.csv", "products.csv"]print(len(files))
```

Output

3

Why?

Because there are three files:

customers.csv

orders.csv

products.csv

AWS Data Engineering example:

```
s3_files = [    "customers.csv",    "orders.csv",    "products.csv",
"sales.csv"]print("Total files:", len(s3_files))
```

Output

Total files: 4

---

5. Adding Data to List – append()

Suppose initially we have:

```
files = ["customers.csv", "orders.csv"]print(files)
```

Output:

```
['customers.csv', 'orders.csv']
```

Now another file arrives:

products.csv

```
We can add it using append().
files.append("products.csv")print(files)
```

Output

```
['customers.csv', 'orders.csv', 'products.csv']
append() means:
```

Add this value to the end of my list.

Data Engineering example:

```
processed_files =
[]processed_files.append("customers.csv")processed_files.append("orders.cs
v")processed_files.append("sales.csv")print(processed_files)
```

Output

```
['customers.csv', 'orders.csv', 'sales.csv']
```

This type of logic can be useful when tracking files processed by a pipeline.

---

6. Removing Data – remove()

```
We can remove an item using remove().
```

```
files = ["customers.csv", "orders.csv",
"products.csv"]files.remove("orders.csv")print(files)
```

Output

```
['customers.csv', 'products.csv']
```

Here Python searches for:

orders.csv

and removes it.

---

7. Change a Value in a List

Lists are **mutable**.

Mutable simply means:

We can change the data after creating the list.

Example:

```
files = ["customers.csv", "orders.csv", "products.csv"]files[1] =
"sales.csv"print(files)
```

Output

```
['customers.csv', 'sales.csv', 'products.csv']
```

Originally:

```
Index 1 = orders.csv
```

We changed it to:

```
Index 1 = sales.csv
```

---

8. List Slicing

Sometimes we don't need the complete list. We need only some values.

Example:

```
files = [    "customers.csv",    "orders.csv",    "products.csv",
"sales.csv"]print(files[0:2])
```

Output

```
['customers.csv', 'orders.csv']
```

0:2 means:

Start from index 0

Stop before index 2

So index 0 and 1 are returned.

---

9. Loop Through a List

```
This is very important for Data Engineering.
```

Suppose we have 3 files:

```
files = [    "customers.csv",    "orders.csv",    "products.csv"]
```

Instead of writing:

```
print(files[0])print(files[1])print(files[2])
```

we can use a loop:

```
for file in files:    print(file)
```

Output

customers.csv

orders.csv

products.csv

Understand this carefully.

```
for file in files:
```

means:

Take one item at a time from files and temporarily store it in file.

First round:

```
file = customers.csv
```

Second round:

```
file = orders.csv
```

Third round:

```
file = products.csv
```

That's why all three are printed.

---

10. AWS Data Engineering Example with List

Imagine files are coming into an S3 bucket.

```
s3_files = [    "customers.csv",    "orders.csv",    "products.csv"]for
file in s3_files:    print("Processing file:", file)
```

Output

Processing file: customers.csv

Processing file: orders.csv

Processing file: products.csv

Later, when learning Boto3, we can replace this sample list with actual objects coming from S3.

---

11. Tuple

A **tuple** is also used to store multiple values.

Example:

```
regions = ("ap-south-1", "us-east-1", "us-west-2")print(regions)
```

Output

('ap-south-1', 'us-east-1', 'us-west-2')

Notice the difference:

List uses:

```
[]
```

Tuple uses:

()

For example:

List

```
["Chennai", "Bangalore", "Hyderabad"]
```

Tuple

("Chennai", "Bangalore", "Hyderabad")

Main difference:

List → Can be changed

Tuple → Cannot normally be changed after creation

Example:

```
regions = ("ap-south-1", "us-east-1")print(regions[0])
```

Output

ap-south-1

```
A tuple can be useful for fixed configuration values that should not be
modified accidentally.
```

---

12. Set

A **set** stores unique values.

Suppose:

```
departments = ["IT", "HR", "IT", "Finance", "HR"]print(departments)
```

Output:

```
['IT', 'HR', 'IT', 'Finance', 'HR']
```

There are duplicates.

Now:

```
unique_departments = set(departments)print(unique_departments)
```

The result contains:

IT

HR

Finance

The display order of a set is not guaranteed, so don't depend on a particular order.

This is useful in Data Engineering when we want to identify unique values.

Example:

```
customer_ids = [101, 102, 101, 103, 102, 104]unique_customer_ids =
set(customer_ids)print(unique_customer_ids)
```

The set contains:

101

102

103

104

The duplicate IDs are removed.

---

13. Dictionary

Now this is **very important for AWS Data Engineering.**

A dictionary stores information in:

KEY : VALUE

format.

Example:

```
customer = {    "id": 101,    "name": "Arun",    "city":
"Chennai"}print(customer)
```

Output

```
{'id': 101, 'name': 'Arun', 'city': 'Chennai'}
```

Understand this:

Key Value

id 101

name Arun

city Chennai

So:

"id": 101

means:

id → Key

101 → Value

---

14. Access Dictionary Values

We can access data using the key.

```
customer = {    "id": 101,    "name": "Arun",    "city":
"Chennai"}print(customer["id"])print(customer["name"])print(customer["city
"])
```

Output

101

Arun

Chennai

Notice the difference.

With a list we normally access using an index:

```
files[0]
```

With a dictionary:

```
customer["name"]
```

we access using a **key**.

---

15. Add a New Key and Value

Initially:

```
customer = {    "id": 101,    "name": "Arun"}
```

Now add city:

```
customer["city"] = "Chennai"print(customer)
```

Output

```
{'id': 101, 'name': 'Arun', 'city': 'Chennai'}
```

---

16. Change Dictionary Value

```
customer = {    "id": 101,    "name": "Arun",    "city":
"Chennai"}customer["city"] = "Bangalore"print(customer)
```

Output

```
{'id': 101, 'name': 'Arun', 'city': 'Bangalore'}
```

So dictionaries are also mutable.

---

17. AWS S3 Example Using Dictionary

Imagine we want to store information about an S3 file.

```
s3_file = {    "bucket_name": "company-raw-data",    "file_name":
"customers.csv",    "record_count": 1000,    "status":
"SUCCESS"}print(s3_file["bucket_name"])print(s3_file["file_name"])print(s3
_file["record_count"])print(s3_file["status"])
```

Output

company-raw-data

customers.csv

1000

SUCCESS

Here:

Variable:

s3_file

Keys:

bucket_name

file_name

record_count

status

Values:

company-raw-data

customers.csv

1000

SUCCESS

---

18. List of Dictionaries

This is one of the most important concepts today.

One customer:

```
customer = {    "id": 101,    "name": "Arun",    "city": "Chennai"}
```

But normally Data Engineers don't process only one customer.

We may have thousands or millions of records.

For learning, take three:

```
customers = [    {        "id": 101,        "name": "Arun",        "city":
"Chennai"    },    {        "id": 102,        "name": "Priya",
"city": "Bangalore"    },    {        "id": 103,        "name": "Kumar",
"city": "Hyderabad"    }]print(customers)
```

Here:

customers → List

Each customer → Dictionary

Structure:

LIST

|

|-- Dictionary → Arun

|

|-- Dictionary → Priya

|

|-- Dictionary → Kumar

This is very common when working with **JSON and API data**.

---

19. Process List of Dictionaries

Now:

```
customers = [    {"id": 101, "name": "Arun", "city": "Chennai"},    {"id":
102, "name": "Priya", "city": "Bangalore"},    {"id": 103, "name":
"Kumar", "city": "Hyderabad"}]for customer in customers:
print(customer["name"])
```

Output

Arun

Priya

Kumar

What is happening?

First loop:

```
customer = {"id": 101, "name": "Arun", "city": "Chennai"}
```

Then:

```
customer["name"]
```

returns:

Arun

Second loop returns:

Priya

Third loop returns:

Kumar

---

20. Real Data Engineering Scenario

Suppose our pipeline receives information about three files:

```
files = [    {        "file_name": "customers.csv",        "records":
1000,        "status": "SUCCESS"    },    {        "file_name":
"orders.csv",        "records": 2500,        "status": "SUCCESS"    },
{        "file_name": "products.csv",        "records": 500,
"status": "FAILED"    }]for file in files:    print(
file["file_name"],        file["records"],        file["status"]    )
```

Output

customers.csv 1000 SUCCESS

orders.csv 2500 SUCCESS

products.csv 500 FAILED

Now we can process only successful files:

```
for file in files:    if file["status"] == "SUCCESS":
print("Processing:", file["file_name"])
```

Output

Processing: customers.csv

Processing: orders.csv

products.csv is not processed because:

```
status = FAILED
```

This starts looking like actual Data Engineering logic:

Receive files

↓

Check file status

↓

SUCCESS?

↓ ↓

Yes No

↓ ↓

Process Skip/Handle Error

---

21. List vs Tuple vs Set vs Dictionary

This is the main thing students should remember:

Type	Example	Main Purpose
List	["a.csv", "b.csv"]	Multiple ordered values
Tuple	("dev", "prod")	Fixed collection
Set	{"IT", "HR"}	Unique values
Dictionary	{"name": "Arun"}	Key-value data

Easy Data Engineering examples:

```
# LISTfiles = ["customers.csv", "orders.csv"]# TUPLEenvironments = ("dev",
"test", "prod")# SETunique_regions = {"Chennai", "Bangalore",
"Hyderabad"}# DICTIONARYfile_info = {    "file": "customers.csv",
"records": 1000}
```

---

22. Practice 1 – S3 Files

Write:

```
files = [    "customers.csv",    "orders.csv",    "sales.csv",
"products.csv"]print("Total files:", len(files))for file in files:
print("Processing:", file)
```

Expected Output

Total files: 4

Processing: customers.csv

Processing: orders.csv

Processing: sales.csv

Processing: products.csv

---

23. Practice 2 – Customer Dictionary

```
customer = {    "customer_id": 1001,    "name": "Ravi",    "city":
"Chennai",    "active": True}print("Customer ID:",
customer["customer_id"])print("Name:", customer["name"])print("City:",
customer["city"])print("Active:", customer["active"])
```

Expected Output

Customer ID: 1001

Name: Ravi

City: Chennai

Active: True

---

24. Practice 3 – Data Pipeline

```
pipeline = {    "pipeline_name": "customer_etl",    "source": "S3",
"target": "Redshift",    "records": 5000,    "status":
"SUCCESS"}print("Pipeline:", pipeline["pipeline_name"])print("Source:",
pipeline["source"])print("Target:", pipeline["target"])print("Records:",
pipeline["records"])print("Status:", pipeline["status"])
```

Expected Output

Pipeline: customer_etl

Source: S3

Target: Redshift

Records: 5000

Status: SUCCESS